CSE 2421

The Stack

Stack

- The stack is the memory set aside as scratch space for a thread of execution.
- The stack is always reserved in a LIFO order; the most recently reserved block is always the next block to be freed.
- Stack is really simple to keep track of; freeing a block from the stack is nothing more than adjusting one pointer.

Stack

- When a function is called, a block is reserved on the top of the stack for all local variables
- When that function returns, the block becomes unused and can be used the next time a function is called.
- Size of stack for each function is built in at compile time

Stack

- Each process running on your system will have its own stack
  - Sometimes each individual thread will too. Depends on the setup
- Remember the types of variables stored on the stack
  - Input function parameters
  - Variables declared inside of a logic block
    - Unless the ‘static’ keyword was used to upgrade it

The Stack

- Stored in computer RAM
- Variables created on the stack will go out of scope and automatically deallocate.
  - NOTE: Deallocate does not mean the value is reset
- Much faster to allocate in comparison to variables on the heap.
- Implemented with an actual stack data structure.
  - FILO
- Stores local data, return addresses, used for parameter passing
- You would use the stack if you know exactly how much data you need to allocate before compile time and it is not too big.

Stack example

```
void doSomething() {
ch c = 'S';
.......
}
int main() {
int x = 0;
int y = 10;
IP -> doSomething();
    someOtherStuff();
}
```

address	value
99 (y)	10
100 (x)	0

Stack example

```
IP -> void doSomething() {
      ch c = 'S';
      .......
      }
      int main() {
      int x = 0;
      int y = 10;
      doSomething();
         someOtherStuff();
      }
```

address	value
98 (c)	'S'
99 (y)	10
100 (x)	0

Stack example

```
void doSomething() {
ch c = 'S';
.......
}
int main() {
int x = 0;
int y = 10;
doSomething();
someOtherStuff();
}
```

IP=>

address	value
~~98 (c)~~ deallocated	'S'
99 (y)	10
100 (x)	0

Common Stack error

```
int* doSomething() {
        ch c = 'S';
        int w = 100;
        //.......
        return &w;
}
int main() {
IP -> int x = 0;
        int y = 10;
        {
                int *z = doSomething();
                printf("%d", *z);
        }
```

address	value
99 (y)	10
100 (x)	0

Common Stack error

```
int* doSomething() {
        char c = 'S';
        int w = 100;
        //.......
IP ->   return &w;
}
int main() {
        int x = 0;
        int y = 10;
        {
                int *z = doSomething();
                printf("%d", *z);
        }
```

address	value
97 (w)	100
98 (c)	'S'
99 (y)	10
100 (x)	0

Common Stack error

```
int* doSomething() {
        ch c = 'S';
        int w = 100;
        //.......
        return &w;
}
int main() {
        int x = 0;
        int y = 10;
        {
    IP ->  int *z = doSomething();
           printf("%d", *z);
        }
//Dosomething returns the address of w which is 97
```

address	value
97 ~~(w)~~	100
98 ~~(c)~~	'S'
99 (y)	10
100 (x)	0

Common Stack error

```
int* doSomething() {
        ch c = 'S';
        int w = 100;
        //.......
        return &w;
}
int main() {
        int x = 0;
        int y = 10;
        {
                int *z = doSomething();
    IP ->       printf("%d", *z);
        }
//Dosomething returns the address of w which is 97
```

address	value
~~97~~	100
98 (z)	97
99 (y)	10
100 (x)	0

Common Stack error

```
int* doSomething() {
    ch c = 'S';
    int w = 100;
    //.......
    return &w;
}
int main() {
    int x = 0;
    int y = 10;
    {
        int *z = doSomething();
        printf("%d", *z);
IP -> }
```

address	value
~~97~~	100
98 (z)	97
99 (y)	10
100 (x)	0

// Prints 100, as desired. but notice how that memory is no longer 'allocated'?

Common Stack error

```
int* doSomething() {
        ch c = 'S';
        int w = 100;
        //.......
        return &w;
}
int main() {
        int x = 0;
        int y = 10;
        {
                int *z = doSomething();
    IP ->       {int q = 40;}
                printf("%d", *z);
        }
// What if we add another statement?
```

address	value
~~97~~	100
98 (z)	97
99 (y)	10
100 (x)	0

Common Stack error

```
int* doSomething() {
        ch c = 'S';
        int w = 100;
        //.......
        return &w;
}
int main() {
        int x = 0;
        int y = 10;
        {
                int *z = doSomething();
                {int q = 40;}
    IP ->       printf("%d", *z);
        }
// What if we add another statement?
```

address	value
97 (q)	40
98 (z)	97
99 (y)	10
100 (x)	0

Common Stack error

```
int* doSomething() {
    ch c = 'S';
    int w = 100;
    //.......
    return &w;
}
int main() {
    int x = 0;
    int y = 10;
    {
        int *z = doSomething();
        {int q = 40;}
        printf("%d", *z); // prints 40, not desired!
IP -> }
```

address	value
97 (q)	40
98 (z)	97
99 (y)	10
100 (x)	0

// Our address was de-allocated and the stack space reused by q

CSE 2421

The Heap

Heap

- The heap is memory set aside for dynamic allocation.
- Unlike the stack, there's no enforced pattern to the allocation and deallocation of blocks from the heap; you can allocate a block at any time and free it at any time.
- Memory is only free’d/deallocated when you explicitly say so

Heap

- This makes it much more complex to keep track of which parts of the heap are allocated or free at any given time; there are many custom heap allocators available to tune heap performance for different usage patterns
  - This can lead to memory fragmentation

The Heap

- Stored in computer RAM just like the stack.
- Variables on the heap must be destroyed manually and never fall out of scope.
- Slower to allocate in comparison to variables on the stack.
- Used on demand to allocate a block of data for use by the program.
- Can have fragmentation when there are a lot of allocations and deallocations

Heap

- Data created on the heap will be pointed to by pointers and allocated with calloc/malloc
- Can have allocation failures if too big of a buffer is requested to be allocated.
- You would use the heap if you don't know exactly how much data you will need at runtime or if you need to allocate a lot of data.
- Responsible for memory leaks

Dynamic Memory Allocation

- If you don’t know the size of a data structure at compile time it can’t be built into the stack
  - how to do it? dynamically allocate memory at runtime
- ex: reading a file into memory. At compile time you didn’t know how big the file would be.

Dynamic Memory Functions

- Can be found in the stdlib.h library:
  - To allocate space in memory you use
    - void* calloc(size_t nelements, sizt_t bytes)
      - Memory to hold n elements of size bytes each
  - To allocate a memory block you use
    - void* malloc(size_t size)
      - Memory of size bytes
  - To reallocate a memory block
    - void* realloc(void *ptr, size_t new_size);
  - To de-allocate previously allocated memory you use
    - void free(void *pointer)
- Each function is used to initialize a pointer with memory from the heap

Dynamic Memory Functions

- Notice the return types of void*
  - This is because the method needs to be generic to work for all types of pointers
  - You are responsible for casting it to the type you wanted

void* malloc(size_t size)

- Parameters:
  - Size of the memory block in bytes.
- Return value:
  - If the request is successful then a pointer to the memory block is returned.
  - If the function failed to allocate the requested block of memory, a null pointer is returned.
- The memory is <u>not initialized to 0's</u>. It can have whatever values it had previously
- Allocates a contiguous block of memory on the heap

void* calloc(size_t num, size_t size)

- Parameters:
  - num - Number of elements to allocate
  - size - size in bytes of each element
- Return value:
  - Will return a pointer to the memory block. If the request fails, a NULL pointer is returned.
- <u>initializes the memory to 0</u>
- Example:

  - ex: ptr_data = (int*) calloc (a, sizeof(int));

void* realloc(void *ptr,size_t new_size)

- Parameters:
  - *ptr – pointer to the initially allocated block of memory
  - new_size – newly desired size
- Return value:
  - Pointer to the memory block requested. NULL if the request failed
    - Not guaranteed to be the same as the original pointer, might have had to move around in the heap to get new space
- If new size is larger, additional space will be allocated at the end of the structure, it will not be initialized to 0's
- if new size is smaller the previous block will be shortened by the required amount from the end of the previous block
- The original contents are preserved

Difference Between Malloc and Calloc

- The number of arguments. malloc() takes a single argument (memory required in bytes), while calloc() needs two arguments.
- malloc() does not initialize the memory allocated, while calloc() initializes the allocated memory to ZERO.

free

- The free function returns memory to the operating system.
  - free(ptr);
  - After freeing a pointer, it is a good idea to reset it to point to 0.

  - NOTE: When 0 is assigned to a pointer, the pointer becomes a null pointer…in other words, it points to nothing. By doing this, when you do something foolish with the pointer (it happens a lot, even with experienced programmers), you find out immediately (runtime-error) instead of later, when you have done considerable damage.

Free ex

```
int *pi  = malloc(100);
.....
//done using pi
free(pi);
.....
*pi = 1231;

// pi still points into the heap
somewhere, but may have been
reallocated to somebody else.

// So now I'm doing something bad
```

Free ex

```
int *pi  = malloc(100);
.....
//done using pi
free(pi);
pi = NULL; //set to NULL
.....
*pi = 1231;
//now if I goof the program will
crash because a NULL pointer is
considered an invalid address
```

void *

- The void* pointer type is a generic way to hold a pointer
- A void* can never be resolved (i.e you can never access what it's pointing to
- Its able to be generic because the size of an address is always consistent on a machine
  - 32-bit machine -> 32 bit address
  - 64-bit machine -> 64 bit address
- void* x;
  - *x = 1234; //INVALID! the compiler has no idea what type x is pointing to so it doesn’t know how to ‘resolve’ the address.

void* example

```
void* x;
float i=10, *j;
void *k;
k=&i;
j=k;
printf(“%f\n”, *k);  //INVALID, no type info
printf("%f\n", *j);
```

void* ex

void* k = (void*)100;

(char*)k

(short*)k

(int*)k

Address	Value
100	43
101	21
102	42
103	17
104	63
105	35
106	97

Without the knowledge of type the compiler doesn't know how to treat the data. i.e how many bytes to grab

Declaration of Arrays

- An array is a way to store many values under the same name in adjacent memory locations.
- Arrays must be declared before they can be used in the program.
- Standard array declaration is as
  - <type> <name> [<size>];
  - <size> elements of the array, stored using an index from 0 to <size>-1
- Examples
  - double height[10]; // height[0] to height[9]
  - float width[20]; // width[0] to width[19]
  - int storage[9]; // etc
  - char name[20]; // a string!
- First index/subscript=0
- An array location can be used like any other single variable:
  - x = array[3]
  - array[5]=x+y

in memory:											
storage->	**[0]**	**[1]**	**[2]**	**[3]**	**[4]**	**[5]**	**[6]**	**[7]**	**[8]**		
address -->	**+0**	**+4**	**+8**	**+12**	**etc**						

Note that the square brackets [] actually represent an operator (known as *array dereference*) which has a relative priority and associativity.

Common Stack error (revisited)

```
int* doSomething() {
    ch c = 'S';
    int w = 100;
    //.......
    return &w;
}
int main() {
    int x = 0;
    int y = 10;

    int *z = doSomething();
    {int q = 40;}
    printf("%d", *z); // prints 40, not desired!
```

address	value
97 (q)	40
98 (z)	97
99 (y)	10
100 (x)	0

// Our address was de-allocated and the stack space reused by q

Common Error - FIXED

```
int* doSomething() {
     ch c = 'S';
    int *w = malloc(sizeof(int));  // now we malloc *w
    *w = 100;                      // set malloc'ed memory
    // .......                     // return the pointer
    return w;
}
int main() {
    int x = 0;
    int y = 10;
    int *z = doSomething();
    { int q = 40; } // This caused error before
    printf("%d", *z);
     // This will print the desired 100, because w was created on the
heap which never goes out of scope
}
```

Pointer Fun cont.

```
int a = 12;
int *b = &a;
int **c = &b;
```

Expression	Equivalent Expression
a	12
b	&a
*b	a, 12
c	&b
*c	b, &a
**c	*b, a, 12

CSE 2421

arrays[0] vs pointers

(the showdown)

Arrays vs. Pointers

```
int *x;
int x[256];
```

- The RHS use of either identifier `x` operates almost identically; use either as a pointer to one-to-many integers, or deference either to obtain an single integer
- Can't put the either on the LHS (i.e. can't say `x = something`, but can say `x[index] = something`)
- Compiler generates instructions to compute the exact stack-pointer-relative address for any chosen element of the array when using an array; it can do this because it knows it's contiguous
- Compiler generates instructions to compute the pointer-value-relative address for any chosen element of the array when using a pointer

Pointer Dereferencing

- `int x[256];`
- `x[i]`
- Read as: the (`i + 1`)st element of array `x` (indices start at `0`)
- Interpretation: set or get 32 bits of memory interpreted as a signed integer beginning at the memory address computed as `address + (i * sizeof(type))`
- For all examples that follow, assume `x` contains the value `10000`, i.e. `x` refers to the `10000`th byte of memory
- x[5] will manipulate or obtain the 32 bit signed integer beginning at address `(10000 + (5 * 4))`, or `10020`

Pointer Manipulation (+1)

- C allows you to “add-to” or “subtract-from” a pointer by using mathematical operators + and –
- C doesn’t add the addend to the address directly, but performs the following computation:
  - `address = address + (addend * sizeof(type))`
- Interpretation: when you add one to a pointer, it causes the pointer to point to the address of the next contiguous piece of memory of sizeof(type)
- Semantic equivalent: were `x` pointing to the ith element of an array in memory, it’s the equivalent of setting `x` to point to the $(i+1)^{st}$ element
- As discussed, dereferencing an array causes a comparable computation:
  - `&x[i] = x + (i * sizeof(type)`

Array iteration vs pointers

```
#define SIZE 10
int values[SIZE];
int *vp;
int v_index;
for(vp=&values[0]; vp< &values[SIZE];)
    *vp++ = 0;  //pointer
//Equivalent to
for(v_index = 0; v_index< SIZE;)
    values[v_index++] = 0; //subscript
```

Efficiency of Subscript vs Pointer

(1) Pointers

```
for(vp=&values[0]; vp< &values[SIZE];)
    *vp++ = 0;  //pointer
```

(2) Subscript

```
for(v_index = 0; v_index< SIZE;)
    values[v_index++] = 0; //subscript
```

How is the iterator working?

(1) At build time the compiler knows that ‘v++’ will also equate to adding 4, so it is replaced with v+=4. So each iteration is a one addition

(2) Every iteration the offset from the beginning of the array has to be recalculated by taking v_index*sizeof(int). Then v_index must be incremented. So each iteration consists of a multiplication and an addition

Relational Operators

- Relational operations on pointers are allowed to compare two pointer values

  < <= > >= !=

- The behavior of the relational operators is only relevant for pointers within the same data structure
- Comparing two arbitrary pointers is undefined behavior

Pointer Manipulation Ex

```
uint32_t A = 112;
uint32_t *E = &A;

printf("%d, ", E);
```

Variable	A	B	C	D	E
Address	x100	x104	x108	x10C	x110
Value	112	08	104	00	100

Pointer Manipulation Ex

```
uint32_t A = 112;
uint32_t *E = &A;

printf("%d, ", E);  // 100
```

Variable	A	B	C	D	E
Address	x100	x104	x108	x10C	x110
Value	112	08	104	00	100

Pointer Manipulation Ex

```
uint32_t A = 112;
uint32_t *E = &A;

printf("%d, ", E);  // 100
E++;
printf("%d ", E);
```

Variable	A	B	C	D	E
Address	x100	x104	x108	x10C	x110
Value	112	08	104	00	100

Pointer Manipulation Ex

```
uint32_t A = 112;
uint32_t *E = &A;

printf("%d, ", E);  // 100
E++;
printf("%d ", E);   // 104
```

Variable	A	B	C	D	E
Address	x100	x104	x108	x10C	x110
Value	112	08	104	00	~~100~~ 104

Implications of the Pointer-Type continued

- Previous example demonstrated operations on 4 byte values and pointer arithmetic by 4-byte values because an `int` is represented as 4 bytes

Implications of the Pointer-Type continued

- If `x` were declared as: `char *x;`
  - Operations on 1 byte signed integers
  - `*x` refers to address `10000`
  - `x[5]` refers to address `10005`
- If `x` were declared as: `short *x;`
  - Operations on 2 byte signed integers
  - `*x` refers to address `10000`
  - `x[5]` refers to address `10010`
- If `x` were declared as `unsigned long long *x;`
  - Operations on 8 byte unsigned integers
  - `*x` refers to address `10000`
  - `x[5]` refers to address `10040`